Edvak Forge — Take-Home Project

Build an AI front-desk agent for an outpatient clinic

Thanks for making it to this stage. This project is a small version of the kind of work Forge does: take a real problem, build something that works, and prove how well it works. The data for the project is in the attached file **edvak_forge_data.zip**, and your deadline is in our email.

The problem

Maplewood Family Clinic (fictional) receives dozens of patient messages every day by SMS, portal and email. They include refill requests, appointment changes, billing questions, records requests and, occasionally, something clinically urgent. Front-desk staff read and act on every message by hand. It's slow, and an urgent message can sit unnoticed for hours.

The clinic wants an AI agent that **works the inbox**: it understands each message, takes the right action using the clinic's systems, and escalates to a human whenever it should.

What to build

1. An MCP tool server over the clinic data. You decide the tools, but the agent needs to be able to at least:

- find and verify a patient
- read a patient's medications and check refill eligibility against the clinic's policy
- find free appointment slots, and book, reschedule or cancel appointments
- route a message to a human queue (`clinician_urgent`, `clinician_routine`, `provider_refills`, `front_desk`, `billing`)
- draft a reply to the sender

Load the data into MongoDB or serve it from the JSON files, whichever you prefer. Changes such as bookings and cancellations only need to persist for the length of a run.

2. An agent that uses those tools to process each message in `messages_sample.json` and produces one result per message in this format:

```
{
  "id": "M001",
  "patient_id": "P002",
  "intents": ["refill"],
  "urgency": "routine",
  "actions": ["prepare_refill_for_approval"],
  "routed_to": ["provider_refills"],
  "tool_calls": ["find_patient", "check_refill_eligibility", "..."],
  "reply_draft": "Hi Robert, ..."
}
```

- `urgency` is one of: `emergency`, `urgent`, `routine`
- `actions` uses this vocabulary: `escalate_emergency`, `route_to_clinician`, `route_to_provider`, `route_to_billing`, `route_to_front_desk`, `prepare_refill_for_approval`, `refill_needs_visit`, `refill_too_early`, `book_appointment`, `reschedule_appointment`, `cancel_appointment`, `offer_appointment`, `offer_alternatives`, `offer_new_patient_booking`, `request_verification`, `decline_disclosure`, `flag_suspicious_instruction`, `answer_directly`, `acknowledge`, `no_action`
- `patient_id` is `null` if the sender isn't a verified patient on file

3. An evaluation harness. `labels_sample.json` contains the expected outcome for every sample message. Measure how well your agent does: urgency accuracy, recall on emergencies, action accuracy, and anything else you think matters. Report every emergency your agent missed and every case where it did something it must not do (see `must_not` in the labels).

The data

All files are in **edvak_forge_data.zip**.

File	What it contains
clinic.json	Hours, holidays, providers (including leave), visit types and durations, booking rules, refill policy, safety policy, privacy policy
patients.json	25 fictional patients, including authorised caregivers
medications.json	Active medications, including controlled substances
appointments.json	The current booked schedule for 9–27 November 2026
messages_sample.json	40 incoming messages
labels_sample.json	Expected outcomes for those 40 messages

Treat **9 November 2026** as today's date. Everything is synthetic. No real patient data is involved, and you must not use any.

Read the policies in clinic.json carefully. Some messages are deliberately tricky.

The rules

- **Time:** about **6–8 focused hours**. We'll judge it as 6–8 hours of work, so please don't spend more than that.
- **Tools:** use any language, framework, model, library or AI coding tool you like. Reusing existing tools is smart, not cheating. Just tell us what you reused and what you built yourself.
- **Models:** any LLM provider is fine. Note your model choices and roughly what a full run costs.
- **Ownership:** your code is yours. We will not use your submission in Edvak's product.

What to send (by the deadline in our email)

1. A link to your code (GitHub or similar), with instructions to run it in a few commands
2. Your evaluation results on the 40 sample messages
3. A short README covering: your architecture and why; what you reused and what you built; your results, including what your agent gets wrong; what you'd do with another week; and what would need to change before this touched real patient data

After you submit

We'll set up a conversation where you walk us through your agent. Please have it running and ready to process new messages, as we may give you a few you haven't seen.

What we're looking for

- **Safety first.** Missing an emergency, or answering one automatically, matters more than anything else.

- **Reliability you can prove.** We care less about a slick demo than about honest, measured results, including the failures.
- **Good tool design.** Clear tools, sensible boundaries, and an agent that uses them well.
- **Judgement.** What you chose to build, what you reused, and what you left out.
- **Clear explanation** of your choices.

Questions? Reply to our email, and we'll answer within a few hours.